

last time

using asymmetric encryption

digital signatures

sign with private key; verify with public key

replay attacks

reflection attacks/mistaken identity

nonces, other context

logic errors evading cryptography

cryptographic hashes; random numbers

random numbers

want keys, etc. to be unguessable and evenly distributed

solution: random numbers

but: many random number functions are not cryptographically secure!

example NOT SECURE: ~~C rand(); Python's random.random~~
better: Python's secrets, os.urandom; Linux getrandom(),
/dev/urandom

extra effort to ensure **not guessable**

need to incorporate “entropy” from unpredictable sources
deliberately unstable circuit; exact timing of input/output; etc.

just asymmetric?

given public-key encryption + digital signatures...

why bother with the symmetric stuff?

symmetric stuff much faster

symmetric stuff much better at supporting larger messages

key agreement

problem: A has B's public encryption key
wants to choose shared secret

some ideas:

- A chooses a key, sends it encrypted to B

- A sends a public key encrypted to B, B chooses a key and sends back

key agreement

problem: A has B's public encryption key
wants to choose shared secret

some ideas:

- A chooses a key, sends it encrypted to B

- A sends a public key encrypted to B, B chooses a key and sends back

alternate model (not needed, but usually used by TLS, SSH, ...):

- both sides generate random values

- derive public-key like "key shares" from values

- use math to combine "key shares"

- kinda like A + B both sending each other public encryption keys

Diffie-Hellman key agreement

A and B want to agree on shared secret

A chooses random value Y

A sends public value derived from Y (“key share”)

B chooses random value Z

B sends public value derived from Z (“key share”)

A combines Y with public value from B to get number

B combines Z with public value from A to get number
and b/c of math chosen, both get same number

Diffie-Hellman key agreement (details, if needed)

math requirement:

some f , so $f(f(X, Y), Z) = f(f(X, Z), Y)$
(that's hard to invert, etc.)

choose X in advance and:

A randomly chooses Y	B randomly chooses Z
A sends $f(X, Y)$ to B	B sends $f(X, Z)$ to A
A computes $f(f(X, Z), Y)$	B computes $f(f(X, Y), Z)$

Diffie-Hellman key agreement (details, if needed)

math requirement:

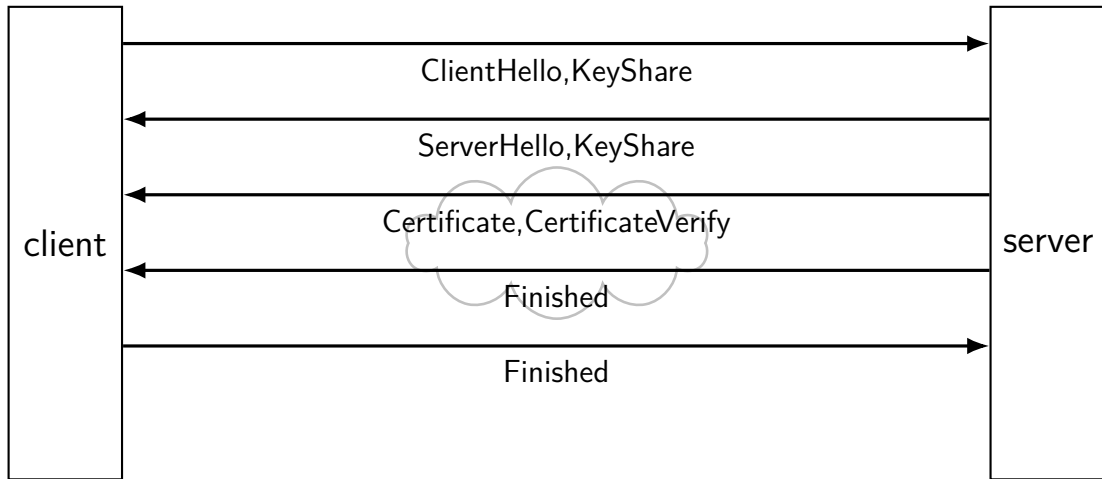
some f , so $f(f(X, Y), Z) = f(f(X, Z), Y)$
(that's hard to invert, etc.)

choose X in advance and:

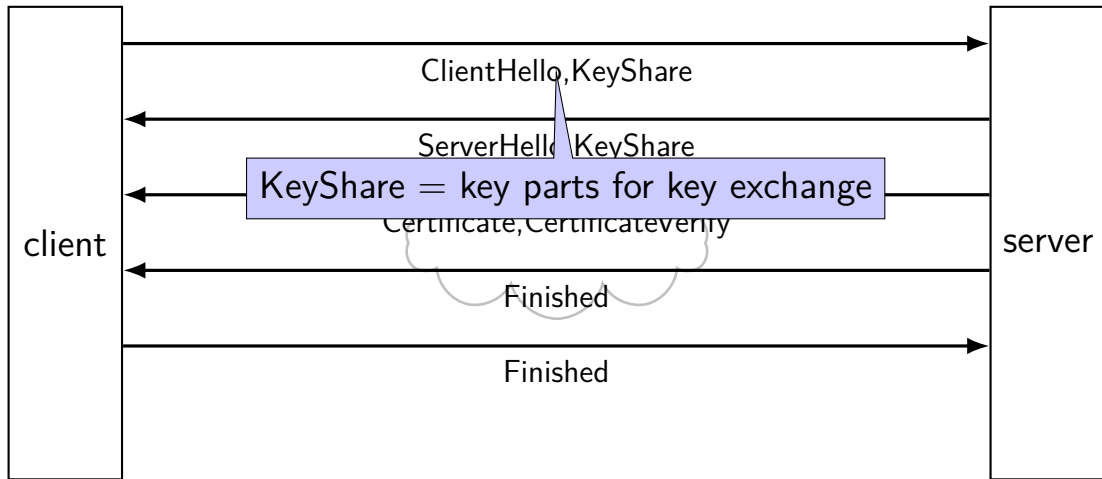
A randomly chooses Y	B randomly chooses Z
A sends $f(X, Y)$ to B	B sends $f(X, Z)$ to A
A computes $f(f(X, Z), Y)$	B computes $f(f(X, Y), Z)$

example $f(a, b) = a^b \pmod{p}$

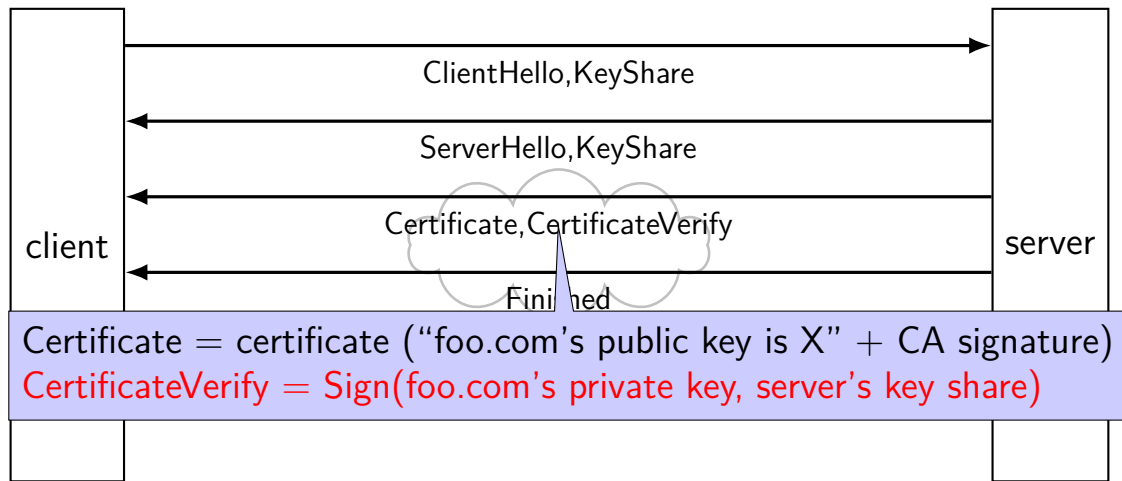
typical TLS handshake



typical TLS handshake



typical TLS handshake



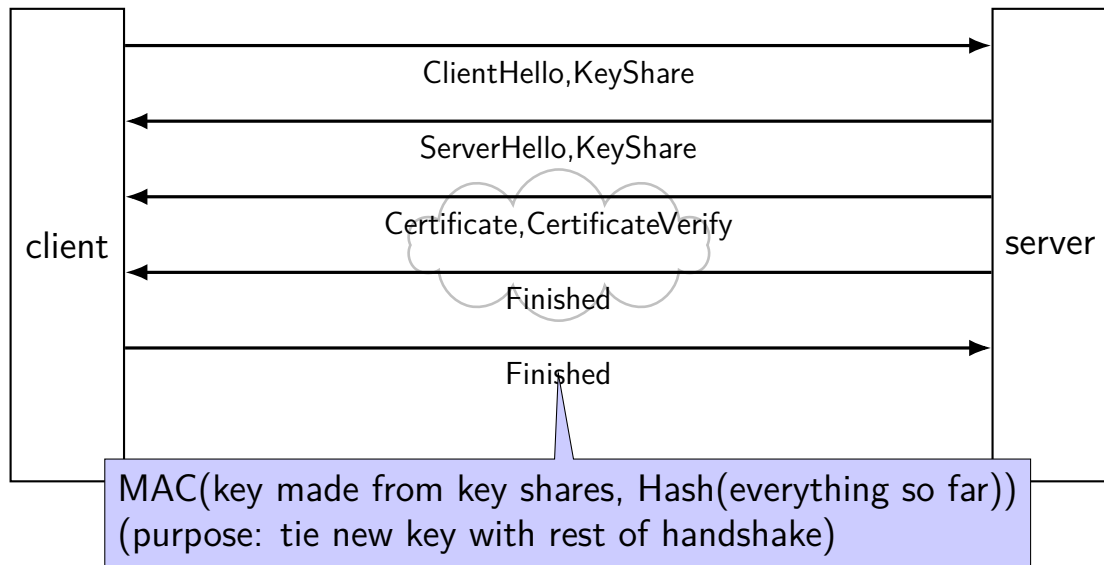
typical TLS handshake



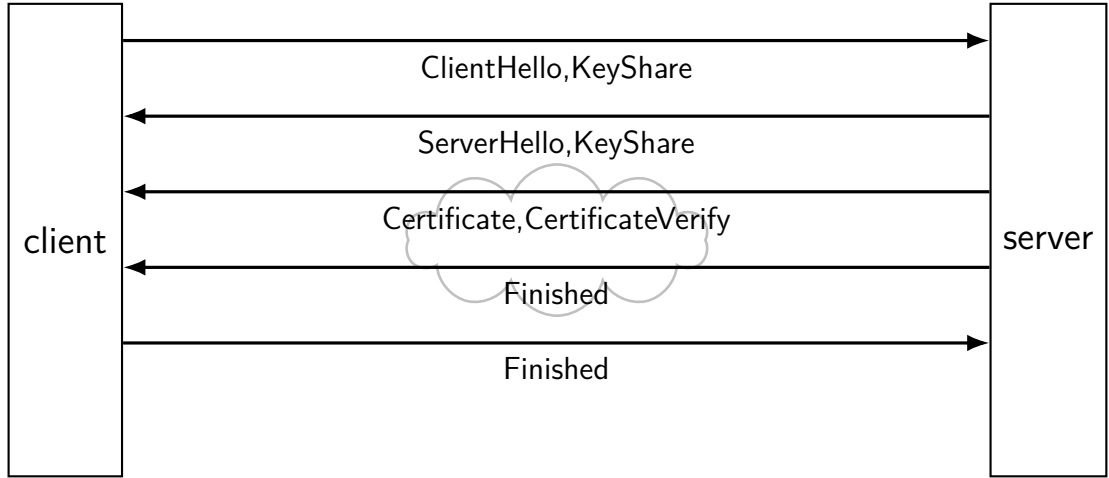
typical TLS handshake



typical TLS handshake



typical TLS handshake



TLS: after handshake

use key shares results to get **several** keys

take $\text{hash}(\text{something} + \text{shared secret})$ to derive each key

separate keys for each direction (server $\rightarrow$ client and vice-versa)

often separate keys for encryption and MAC

later messages use encryption + MAC + nonces

things modern TLS usually does

(not all these properties provided by all TLS versions and modes)

confidentiality/authenticity

- server = one ID'd by certificate

- client = same throughout whole connection

forward secrecy

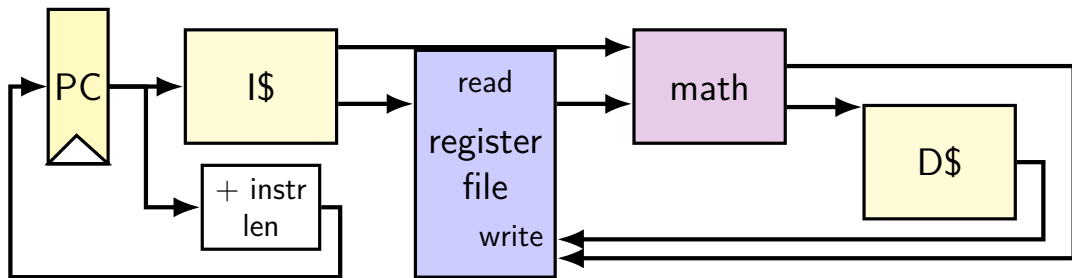
- can't decrypt old conversations (data for KeyShares is temporary)

fast

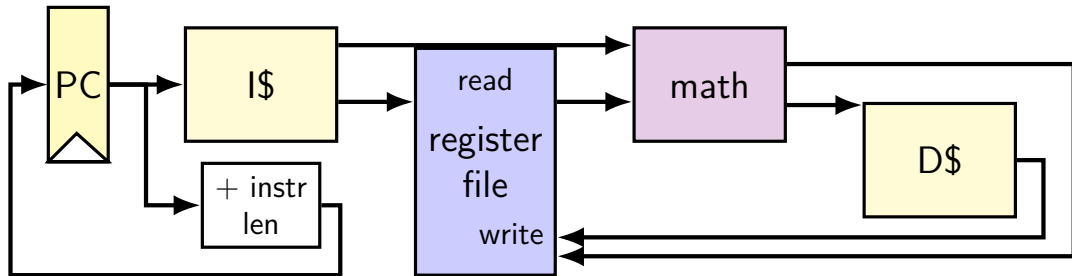
- most communication done with more efficient symmetric ciphers

- 1 set of messages back and forth to setup connection

simple CPU



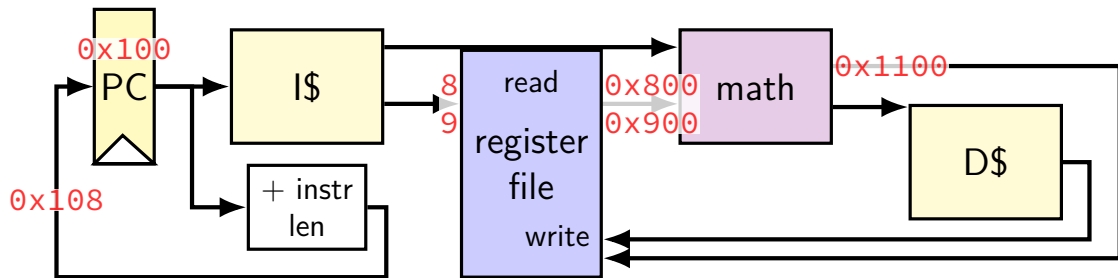
running instructions



```
0x100: addq %r8, %r9
0x108: movq 0x1234(%r10), %r11
```

```
...
%r8: 0x800
%r9: 0x900
%r10: 0x1000
%r11: 0x1100
...
```

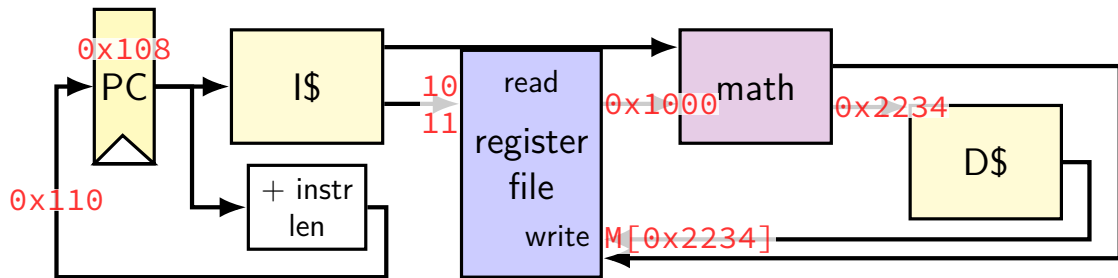
running instructions



```
0x100: addq %r8, %r9
0x108: movq 0x1234(%r10), %r11
```

```
...
%r8: 0x800
%r9: 0x1100
%r10: 0x1000
%r11: 0x1100
...
```

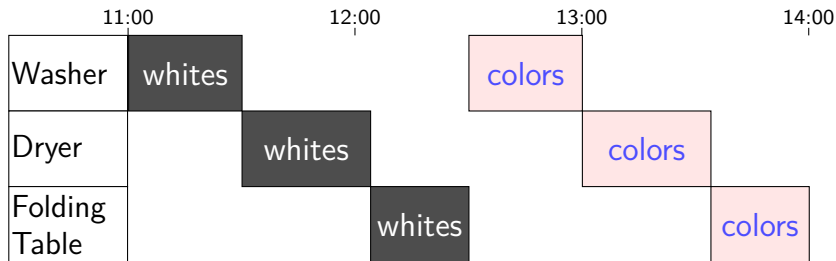
running instructions



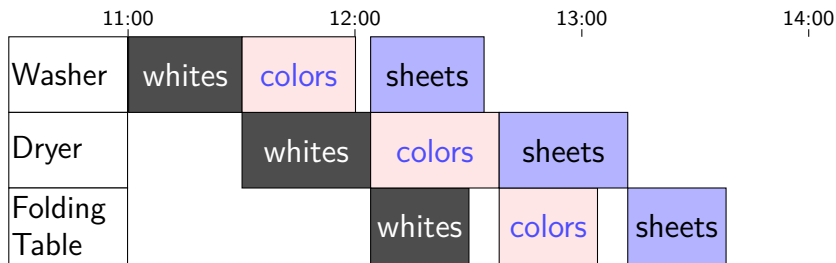
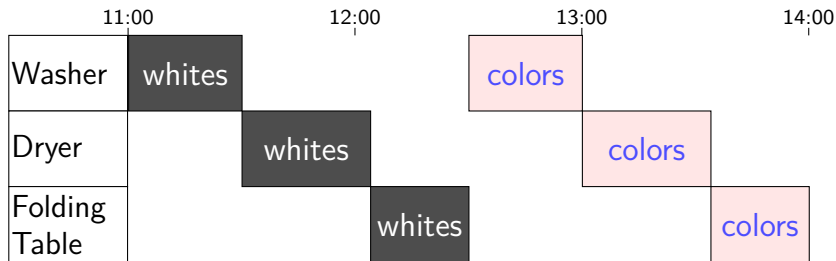
```
0x100: addq %r8, %r9
0x108: movq 0x1234(%r10), %r11
```

```
...
%r8: 0x800
%r9: 0x1100
%r10: 0x1000
%r11: M[0x2234]
...
```

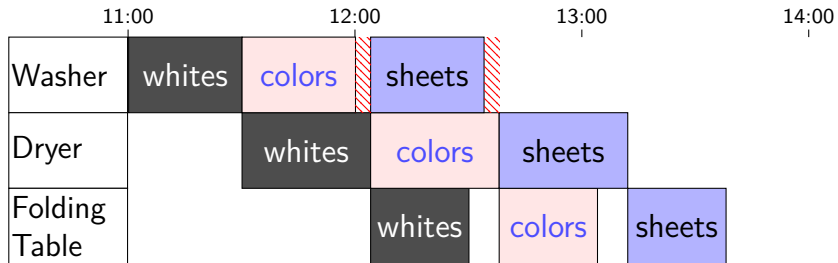
Human pipeline: laundry



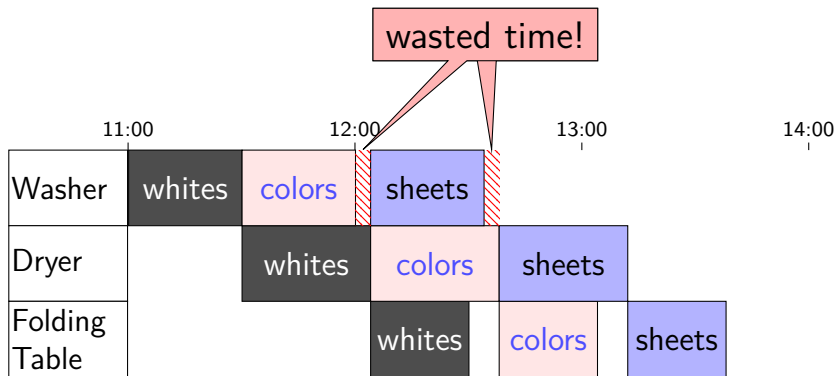
Human pipeline: laundry



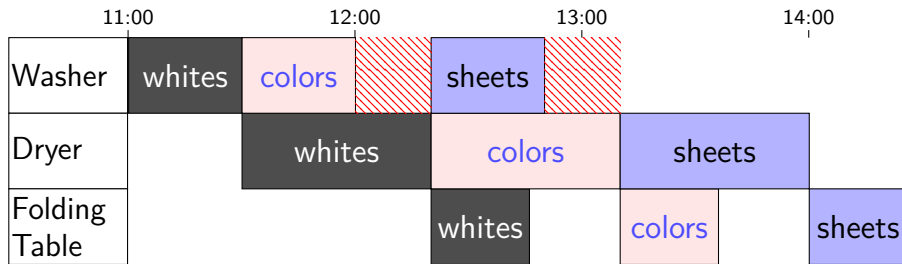
Waste (1)



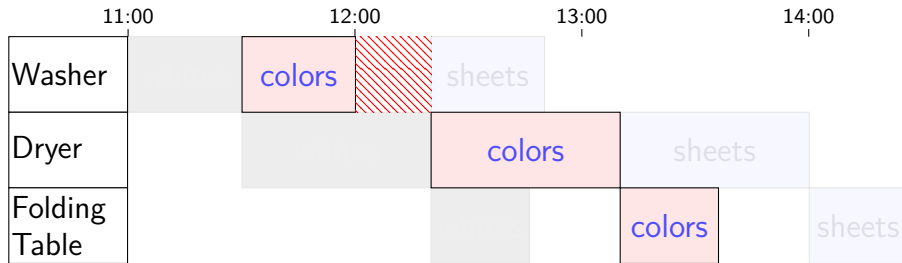
Waste (1)



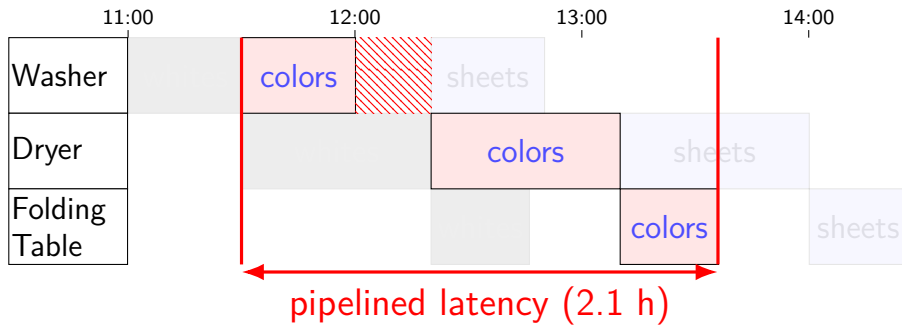
Waste (2)



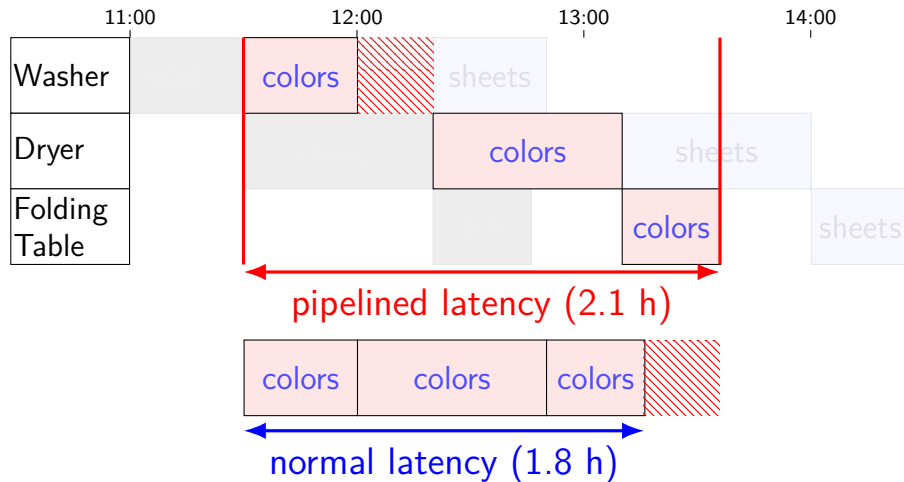
Latency — Time for One



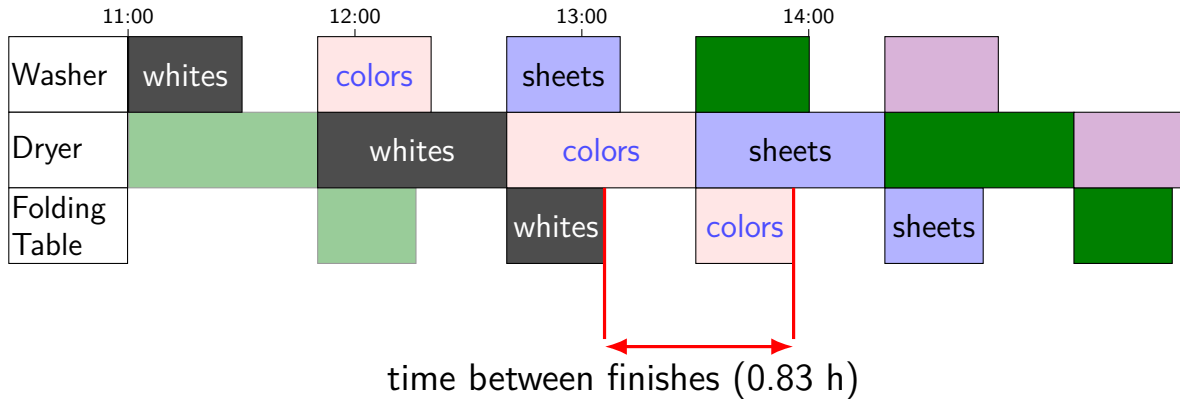
Latency — Time for One



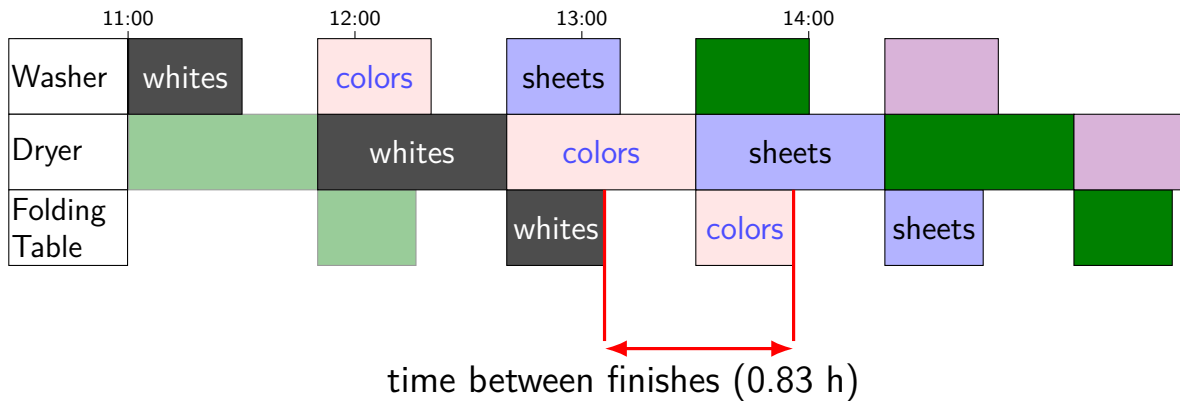
Latency — Time for One



Throughput — Rate of Many

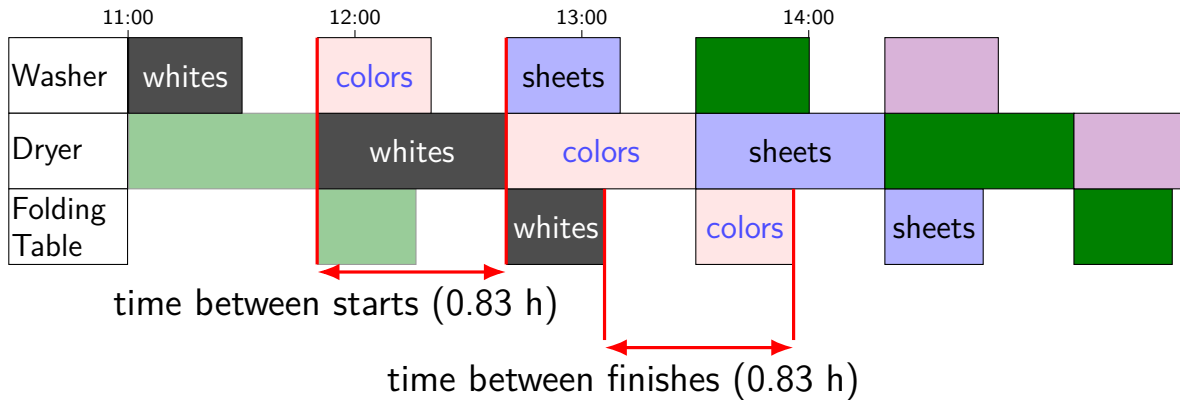


Throughput — Rate of Many



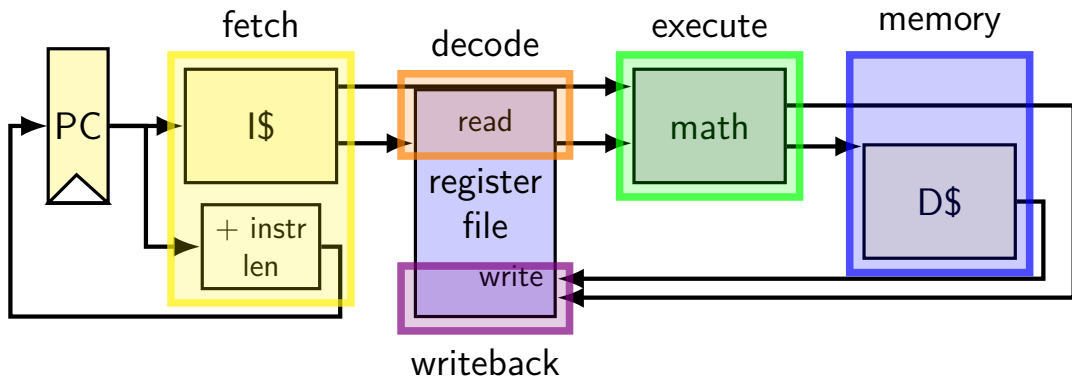
$$\frac{1 \text{ load}}{0.83\text{h}} = 1.2 \text{ loads/h}$$

Throughput — Rate of Many



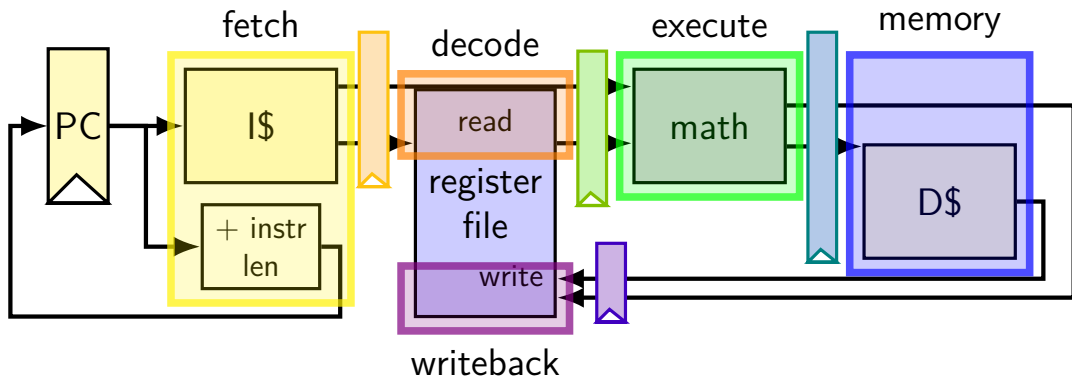
$$\frac{1 \text{ load}}{0.83\text{h}} = 1.2 \text{ loads/h}$$

adding stages (one way)



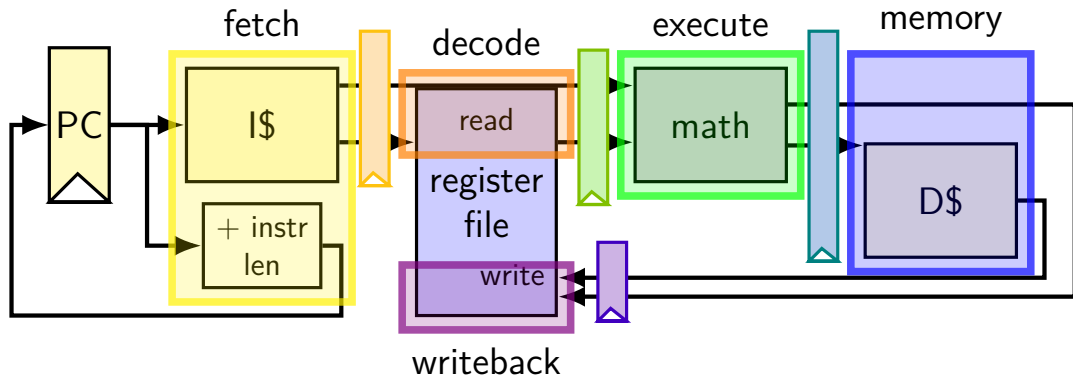
divide running instruction into steps
one way: fetch / decode / execute / memory / writeback

adding stages (one way)

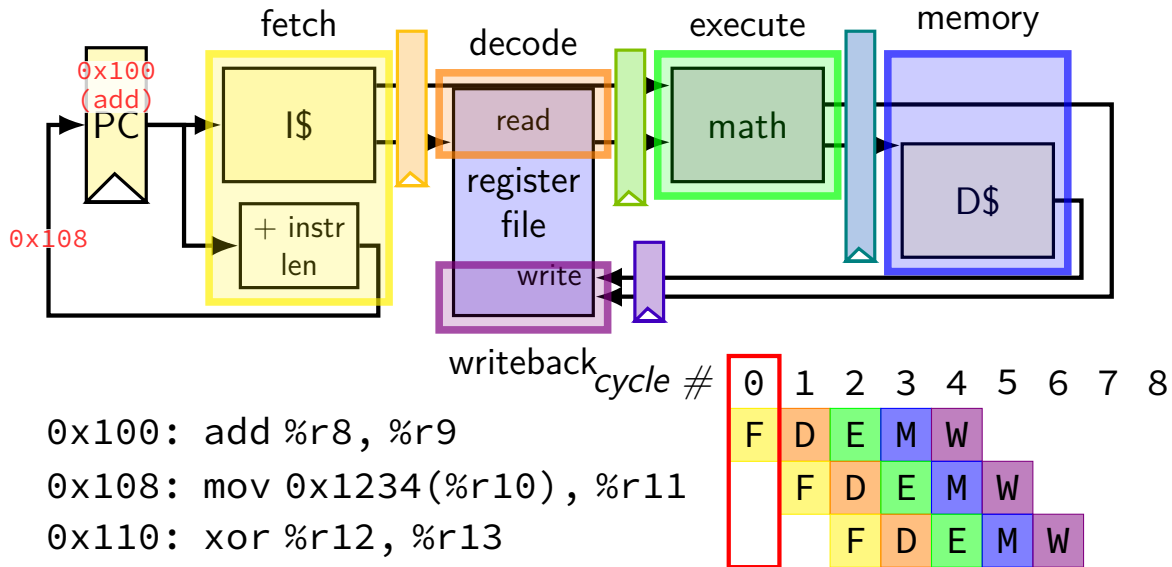


add 'pipeline registers' to hold values from instruction

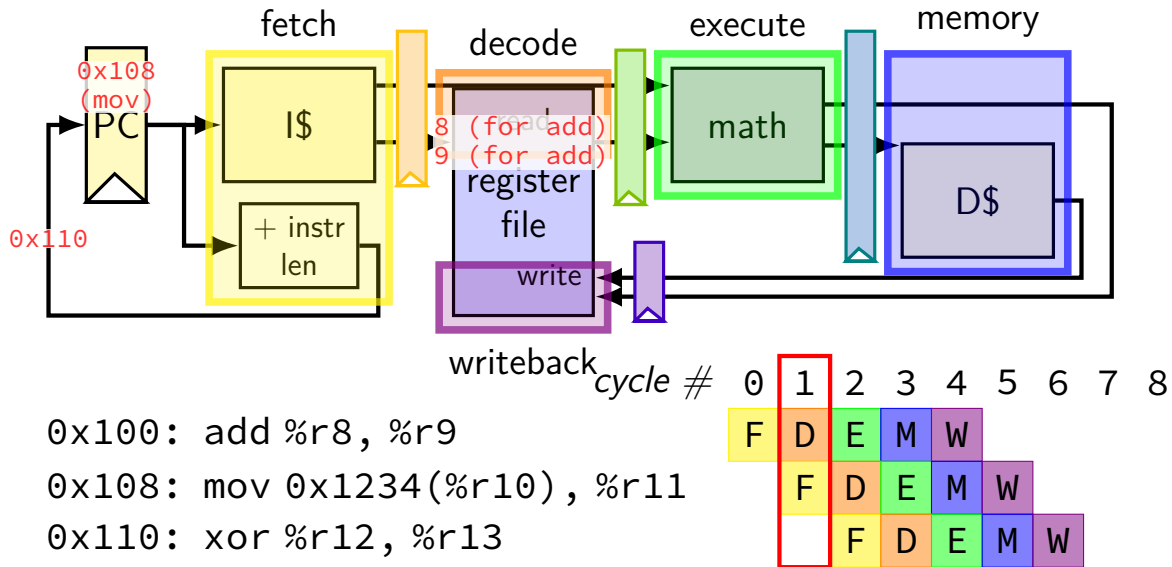
running some instructions



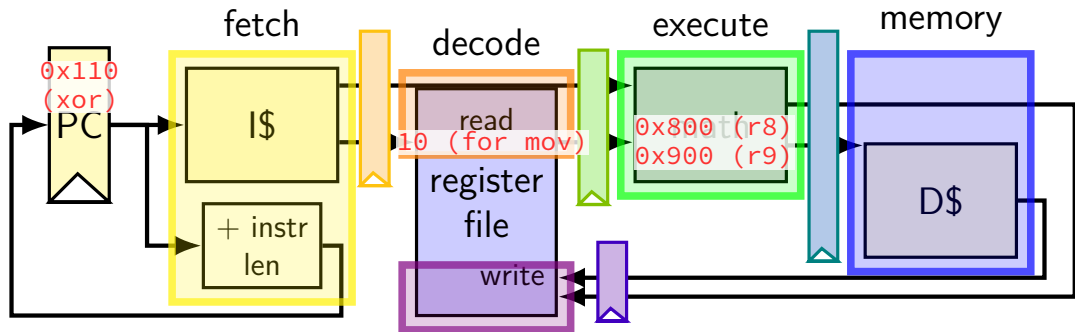
running some instructions



running some instructions



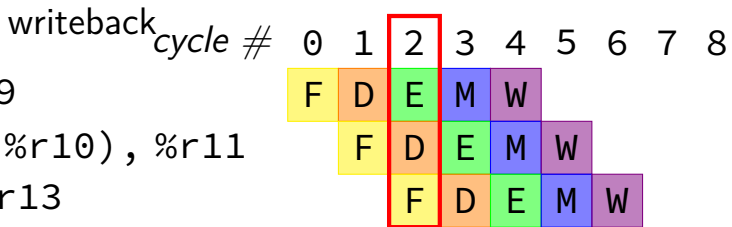
running some instructions



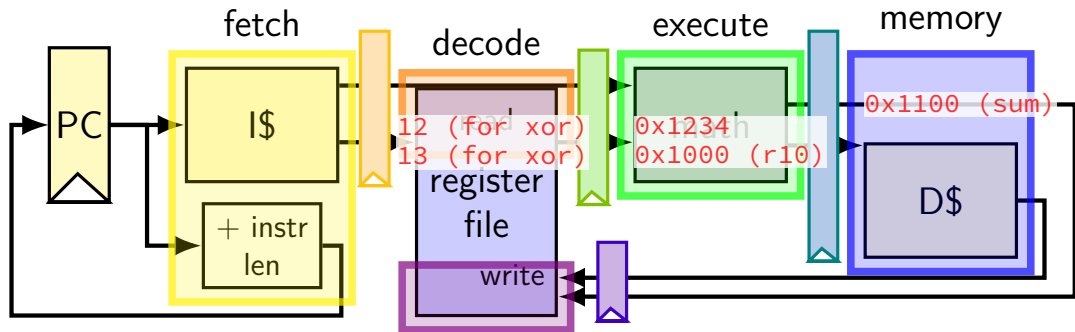
`0x100: add %r8, %r9`

`0x108: mov 0x1234(%r10), %r11`

`0x110: xor %r12, %r13`

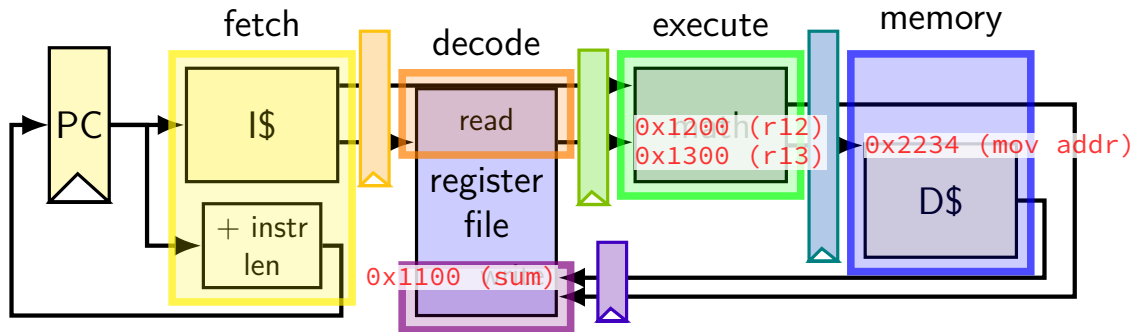


running some instructions



	cycle #	0	1	2	3	4	5	6	7	8
0x100: add %r8, %r9		F	D	E	M	W				
0x108: mov 0x1234(%r10), %r11			F	D	E	M	W			
0x110: xor %r12, %r13				F	D	E	M	W		

running some instructions



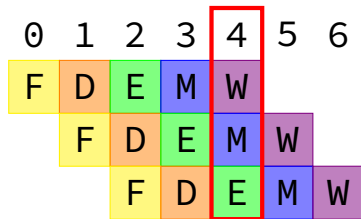
writeback

cycle # 0 1 2 3 4 5 6 7 8

0x100: add %r8, %r9

0x108: mov 0x1234(%r10), %r11

0x110: xor %r12, %r13



why registers?

example: fetch/decode

need to store current instruction somewhere ...while fetching next one

exercise: throughput/latency (1)

	cycle #	0	1	2	3	4	5	6	7	8
0x100: add %r8, %r9		F	D	E	M	W				
0x108: mov 0x1234(%r10), %r11			F	D	E	M	W			
0x110: ...					...					

suppose cycle time is 500 ps

exercise: latency of one instruction?

- A. 100 ps B. 500 ps C. 2000 ps D. 2500 ps E. something else

exercise: throughput/latency (1)

	cycle #	0	1	2	3	4	5	6	7	8
0x100: add %r8, %r9		F	D	E	M	W				
0x108: mov 0x1234(%r10), %r11			F	D	E	M	W			
0x110: ...					...					

suppose cycle time is 500 ps

exercise: latency of one instruction?

- A. 100 ps B. 500 ps C. 2000 ps D. 2500 ps E. something else

exercise: throughput overall?

- A. 1 instr/100 ps B. 1 instr/500 ps C. 1 instr/2000ps D. 1 instr/2500 ps
E. something else

exercise: throughput/latency (2)

0x100: add %r8, %r9
0x108: mov 0x1234(%r10), %r11
0x110: ...

cycle #	0	1	2	3	4
	F	D	E	M	W
		F	D	E	M
				...	

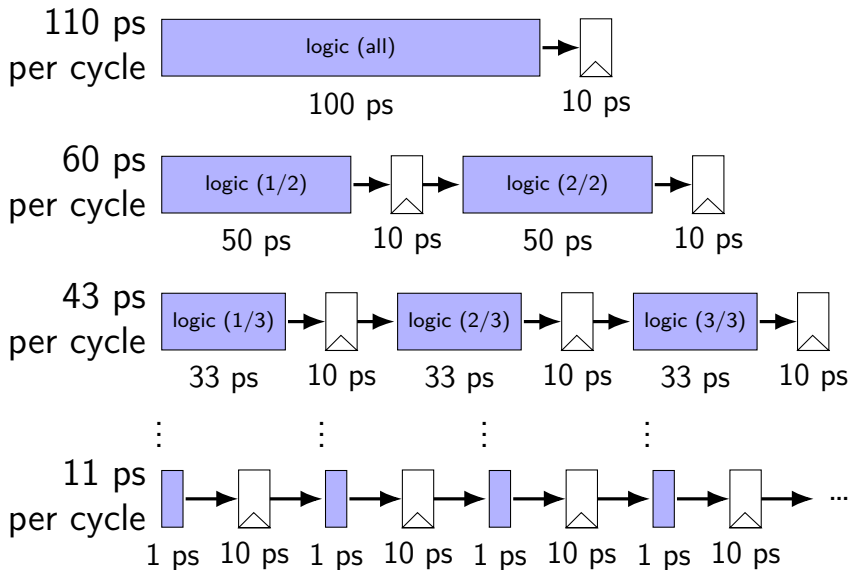
0x100: add %r8, %r9
0x108: mov 0x1234(%r10), %r11
0x110: ...

cycle #	0	1	2	3	4	5	6	7	8
	F1	F2	D1	D2	E1	E2	M1	M2	W1
		F1	F2	D1	D2	E1	E2	M1	M2
				...					

double number of pipeline stages (to 10) + decrease cycle time from 500 ps to 250 ps — throughput?

- A. 1 instr/100 ps B. 1 instr/250 ps C. 1 instr/1000ps D. 1 instr/5000 ps
E. something else

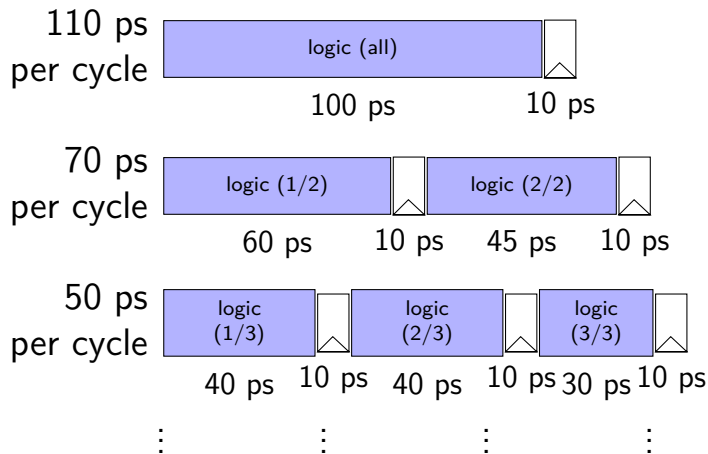
diminishing returns: register delays



diminishing returns: uneven split

Can we split up some logic (e.g. adder) arbitrarily?

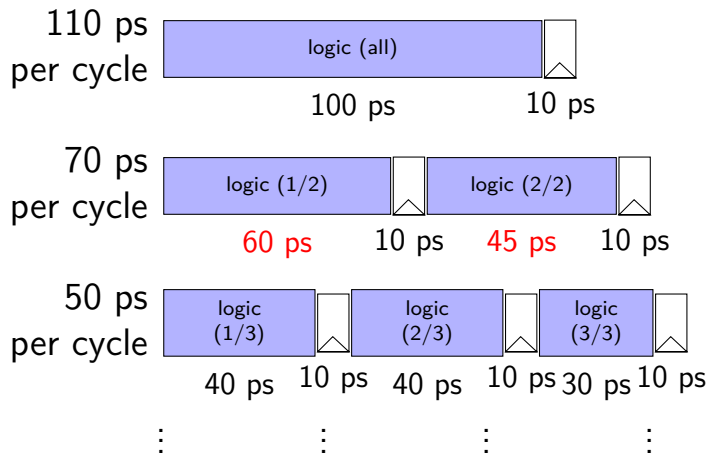
Probably not...



diminishing returns: uneven split

Can we split up some logic (e.g. adder) arbitrarily?

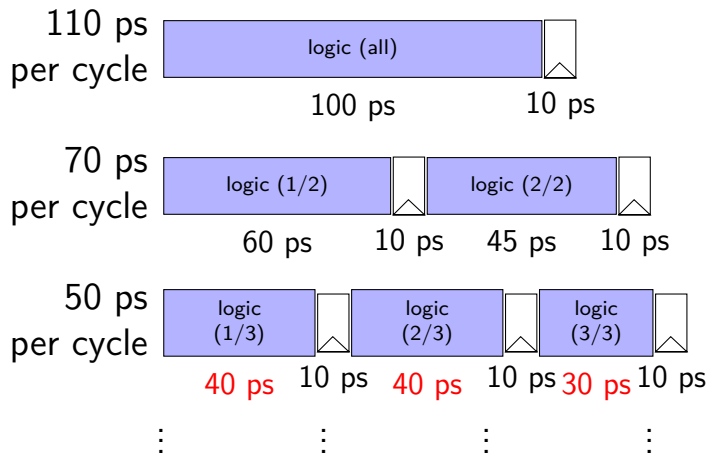
Probably not...



diminishing returns: uneven split

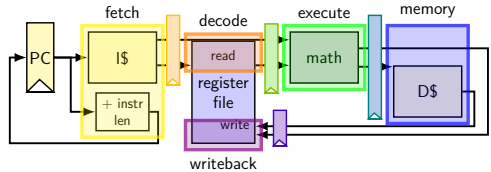
Can we split up some logic (e.g. adder) arbitrarily?

Probably not...



a data hazard

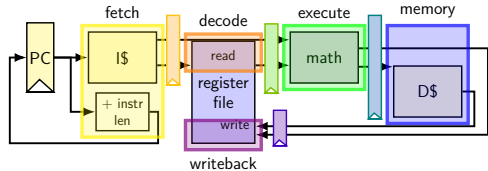
```
// initially %r8 = 800,
//                %r9 = 900, etc.
addq %r8, %r9    // R8 + R9 -> R9
addq %r9, %r8    // R9 + R8 -> R9
addq ...
addq ...
```



	fetch	fetch/decode		decode/execute			execute/memory		memory/writeback	
cycle	PC	rA	rB	R[rB]	R[rB]	rB	sum	rB	sum	rB
0	0x0									
1	0x2	8	9							
2		9	8	800	900	9				
3				900	800	8	1700	9		
4							1700	8	1700	9
5									1700	8

a data hazard

```
// initially %r8 = 800,
//                %r9 = 900, etc.
addq %r8, %r9    // R8 + R9 -> R9
addq %r9, %r8    // R9 + R8 -> R9
addq ...
addq ...
```



	fetch	fetch/decode		decode/execute			execute/memory		memory/writeback	
cycle	PC	rA	rB	R[rB]	R[rB]	rB	sum	rB	sum	rB
0	0x0									
1	0x2	8	9							
2		9	8	800	900	9				
3				900	800	8	1700	9		
4							1700	8	1700	9
5									1700	8

should be 1700

data hazard

addq %r8, %r9 // (1)

addq %r9, %r8 // (2)

step#	pipeline implementation	ISA specification
1	read r8, r9 for (1)	read r8, r9 for (1)
2	read r9, r8 for (2)	write r9 for (1)
3	write r9 for (1)	read r9, r8 for (2)
4	write r8 for (2)	write r8 for (2)

pipeline reads **older value**...

instead of value ISA says was just written

data hazard compiler solution

```
addq %r8, %r9  
nop  
nop  
addq %r9, %r8
```

one solution: **change the ISA**

all addqs take effect **three instructions later**

(assuming can read register value while it is being written back)

make it **compiler's job**

problem: recompile everytime processor changes?

data hazard compiler solution

```
addq %r8, %r9  
nop  
nop  
addq %r9, %r8
```

one solution: **change the ISA**

all addqs take effect **three instructions later**
(assuming can read register value while it is being written back)

make it **compiler's job**

problem: recompile everytime processor changes?

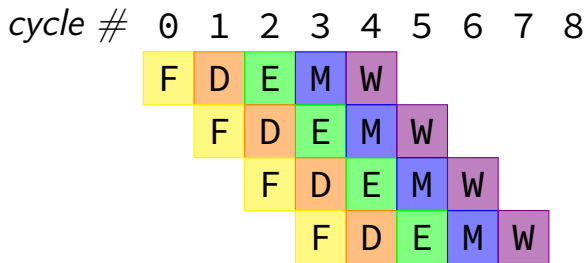
stalling/nop pipeline diagram (1)

add %r8, %r9

nop

nop

addq %r9, %r8



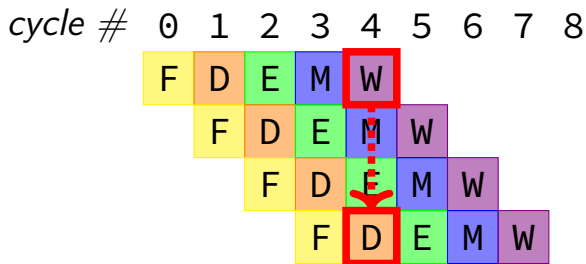
stalling/nop pipeline diagram (1)

add %r8, %r9

nop

nop

addq %r9, %r8



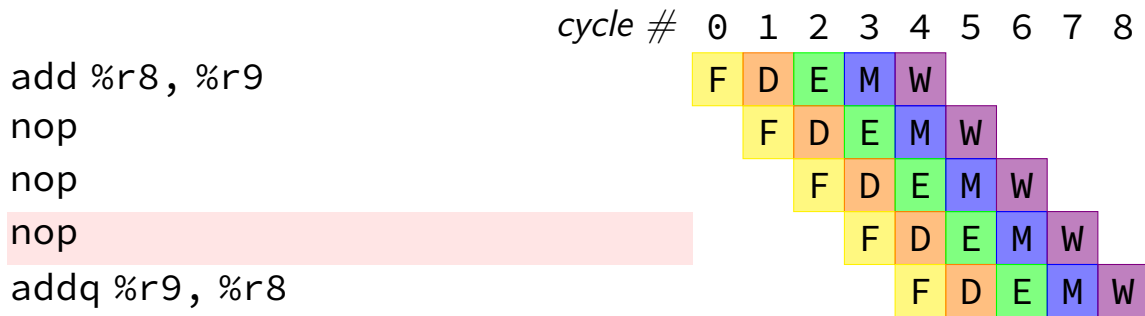
assumption:

if writing register value

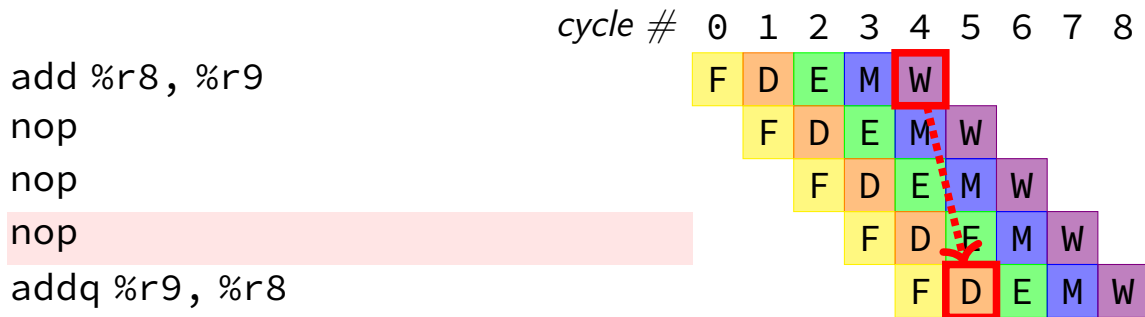
register file will return that value for reads

not actually way register file worked in single-cycle CPU
(e.g. can read old %r9 while writing new %r9)

stalling/nop pipeline diagram (2)



stalling/nop pipeline diagram (2)



if we didn't modify the register file, we'd need an extra cycle

data hazard hardware solution

```
addq %r8, %r9  
// hardware inserts: nop  
// hardware inserts: nop  
addq %r9, %r8
```

how about hardware add nops?

called **stalling**

extra logic:

- sometimes don't change PC

- sometimes put do-nothing values in pipeline registers

control hazard

0x00: cmpq %r8, %r9

0x08: je 0xFFFF

0x10: addq %r10, %r11

	fetch	fetch→decode		decode→execute		execute→write	execute→writeback		...
cycle	PC	rA	rB	R[rA]	R[rB]	result	...	...	...
0	0x0								
1	0x8	8	9						
2	???	---	---	800	900				
3	???	---	---	---	---	less than			

control hazard

0x00: cmpq %r8, %r9

0x08: je 0xFFFF

0x10: addq %r10, %r11

	fetch	fetch→decode		decode→execute		execute→write	execute→writeback		...
cycle	PC	rA	rB	R[rA]	R[rB]	result	...	...	...
0	0x0								
1	0x8	8	9						
2	???	---	---	800	900				
3	???	---	---	---	---	less than			

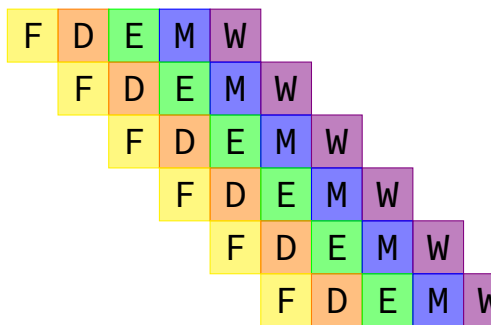
0xFFFF if $R[8] = R[9]$; 0x10 otherwise

jXX: stalling?

```
cmpq %r8, %r9
jne LABEL      // not taken
xorq %r10, %r11
movq %r11, 0(%r12)
```

```
...
cmpq %r8, %r9
jne LABEL
(do nothing)
(do nothing)
xorq %r10, %r11
movq %r11, 0(%r12)
...
```

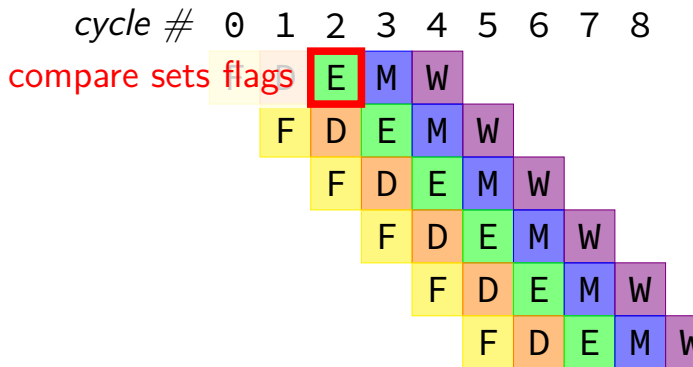
cycle # 0 1 2 3 4 5 6 7 8



jXX: stalling?

```
cmpq %r8, %r9
jne LABEL      // not taken
xorq %r10, %r11
movq %r11, 0(%r12)
```

```
...
cmpq %r8, %r9
jne LABEL
(do nothing)
(do nothing)
xorq %r10, %r11
movq %r11, 0(%r12)
...
```



jXX: stalling?

```
cmpq %r8, %r9
jne LABEL      // not taken
xorq %r10, %r11
movq %r11, 0(%r12)
```

...

```
cmpq %r8, %r9
```

```
jne LABEL
```

```
(do nothing)
```

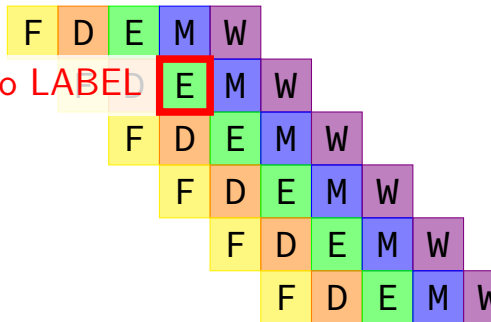
```
(do nothing)
```

```
xorq %r10, %r11
```

```
movq %r11, 0(%r12)
```

...

cycle # 0 1 2 3 4 5 6 7 8

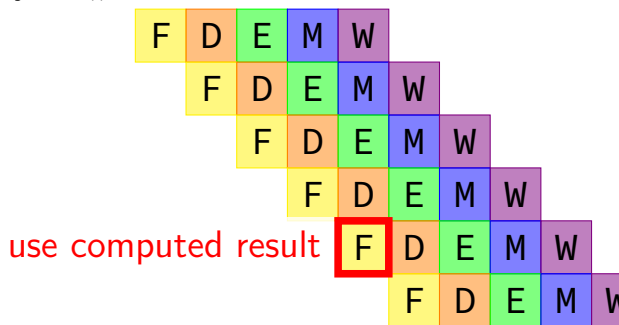


jXX: stalling?

```
cmpq %r8, %r9
jne LABEL      // not taken
xorq %r10, %r11
movq %r11, 0(%r12)
```

```
...
cmpq %r8, %r9
jne LABEL
(do nothing)
(do nothing)
xorq %r10, %r11
movq %r11, 0(%r12)
...
```

cycle # 0 1 2 3 4 5 6 7 8



making guesses

```
    cmpq %r8, %r9  
    jne LABEL  
    xorq %r10, %r11  
    movq %r11, 0(%r12)  
    ...
```

```
LABEL:  addq %r8, %r9  
        imul %r13, %r14  
        ...
```

speculate (guess): **jne** won't go to LABEL

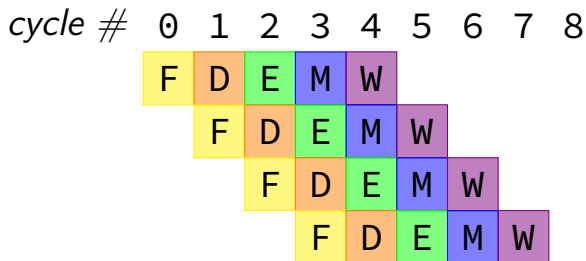
right: 2 cycles faster!; wrong: undo guess before too late

jXX: speculating right (1)

```
cmpq %r8, %r9
jne LABEL
xorq %r10, %r11
movq %r11, 0(%r12)
...
```

```
LABEL:  addq %r8, %r9
        imul %r13, %r14
        ...
```

```
cmpq %r8, %r9
jne LABEL
xorq %r10, %r11
movq %r11, 0(%r12)
...
```



jXX: speculating wrong

cycle # 0 1 2 3 4 5 6 7 8

cmpq %r8, %r9

jne LABEL

xorq %r10, %r11

(inserted nop)

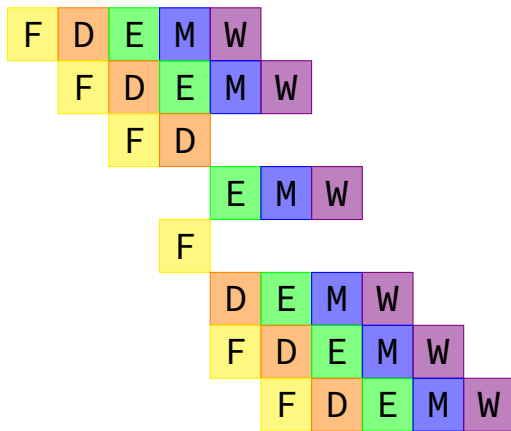
movq %r11, 0(%r12)

(inserted nop)

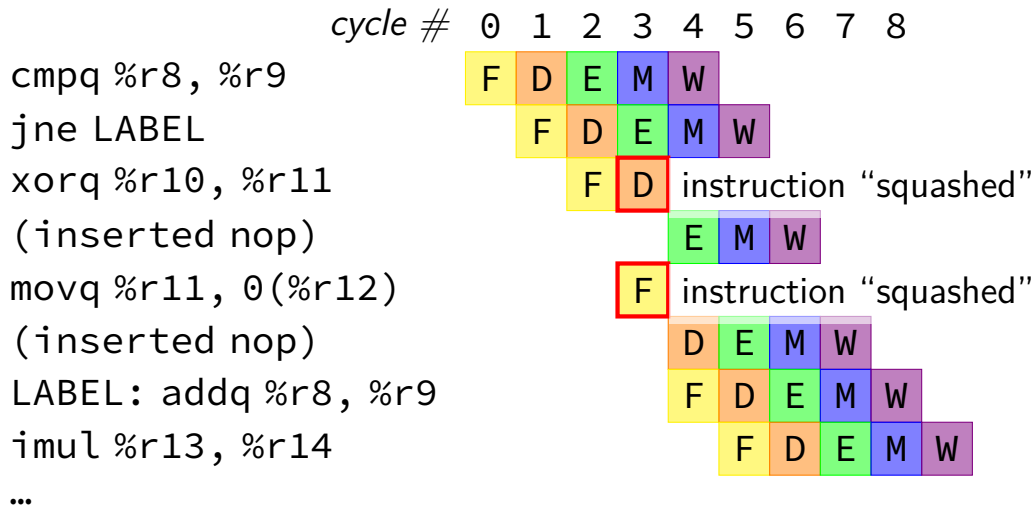
LABEL: addq %r8, %r9

imul %r13, %r14

...



jXX: speculating wrong



“squashed” instructions

on misprediction need to undo partially executed instructions

mostly: remove from pipeline registers

more complicated pipelines: replace written values in
cache/registers/etc.

opportunity

```
// initially %r8 = 800,  
//                %r9 = 900, etc.
```

```
0x0: addq %r8, %r9
```

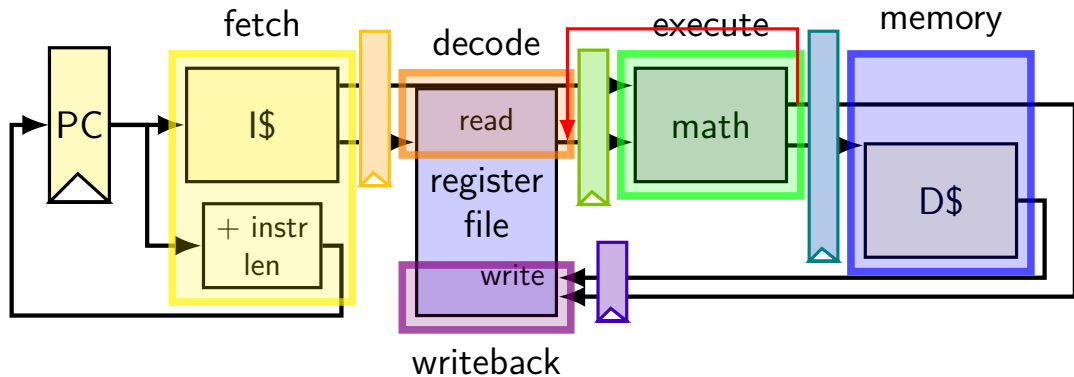
```
0x2: addq %r9, %r8
```

...

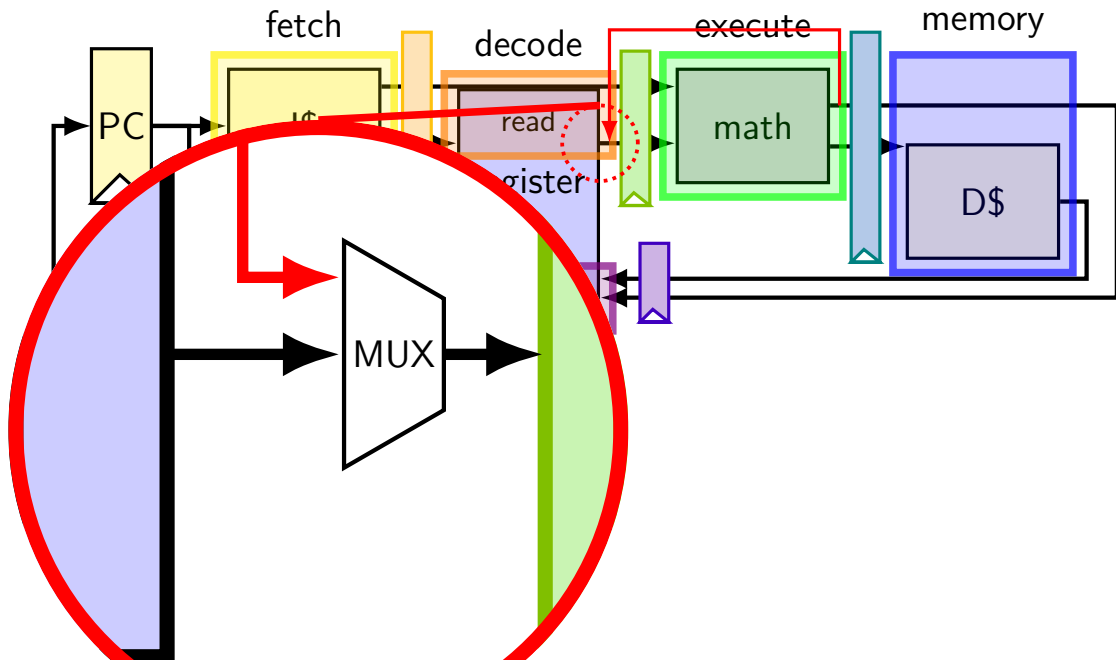
	fetch	fetch/decode		decode/execute			execute/memory		memory/writeback	
cycle	PC	rA	rB	R[rB]	R[rB]	rB	sum	rB	sum	rB
0	0x0									
1	0x2	8	9							
2		9	8	800	900	9				
3				900	800	8	1700	9		
4							1700	8	1700	9
5									1700	8

should be 1700

exploiting the opportunity



exploiting the opportunity



opportunity 2

```
// initially %r8 = 800,  
//                %r9 = 900, etc.
```

```
0x0: addq %r8, %r9
```

```
0x2: nop
```

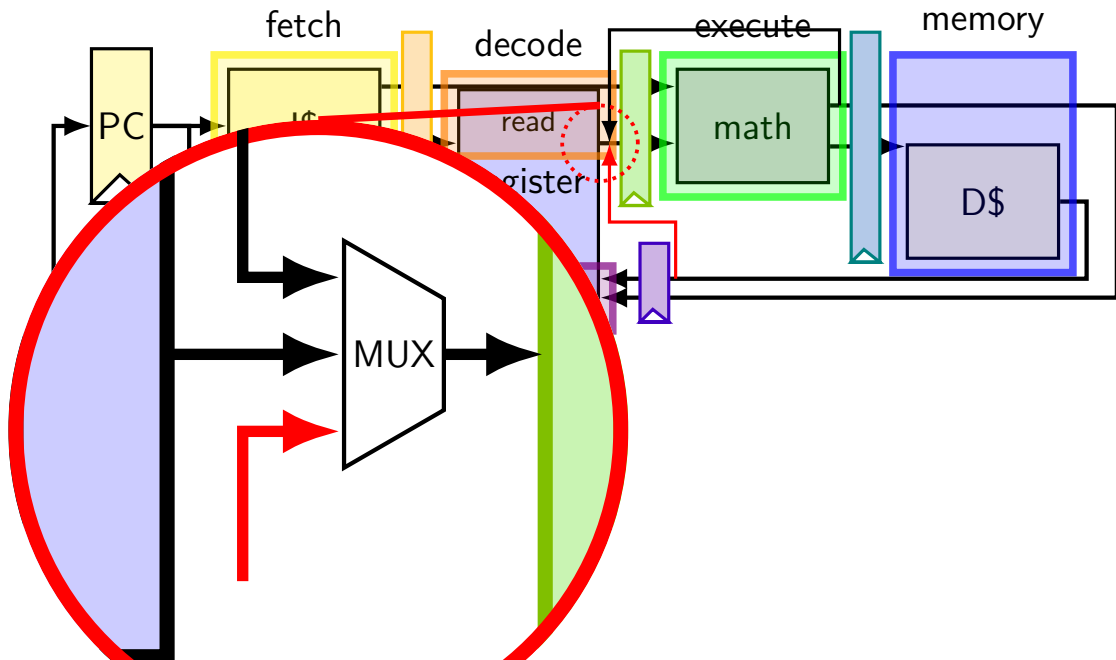
```
0x3: addq %r9, %r8
```

```
...
```

	fetch	fetch/decode		decode/execute			execute/memory		memory/writeback	
cycle	PC	rA	rB	R[rB]	R[rB]	rB	sum	rB	sum	rB
0	0x0									
1	0x2	8	9							
2	0x3	---	---	800	900	9				
3		9	8	---	---	---	1700	9		
4				900	800	8	---	---	1700	9
5							1700	9	---	---
6									1700	9

should be 1700

exploiting the opportunity



exercise: forwarding paths

cycle # 0 1 2 3 4 5 6 7 8

addq %r8, %r9

subq %r8, %r10

xorq %r8, %r9

andq %r9, %r8

F	D	E	M	W				
	F	D	E	M	W			
		F	D	E	M	W		
			F	D	E	M	W	

in subq, %r8 is _____ addq.

in xorq, %r9 is _____ addq.

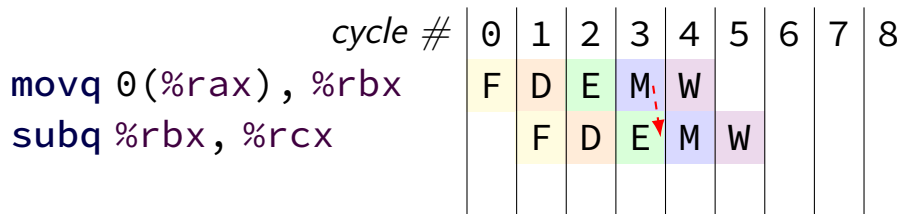
in andq, %r9 is _____ addq.

in andq, %r9 is _____ xorq.

A: not forwarded from

B-D: forwarded to decode from {execute,memory,writeback} stage of

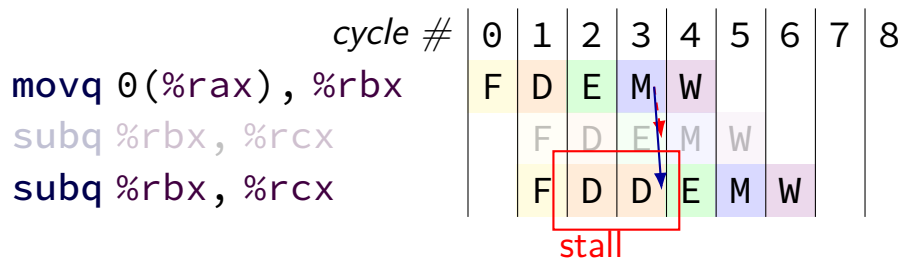
unsolved problem



combine stalling and forwarding to resolve hazard

assumption in diagram: hazard detected in subq's decode stage
(since easier than detecting it in fetch stage)

unsolved problem



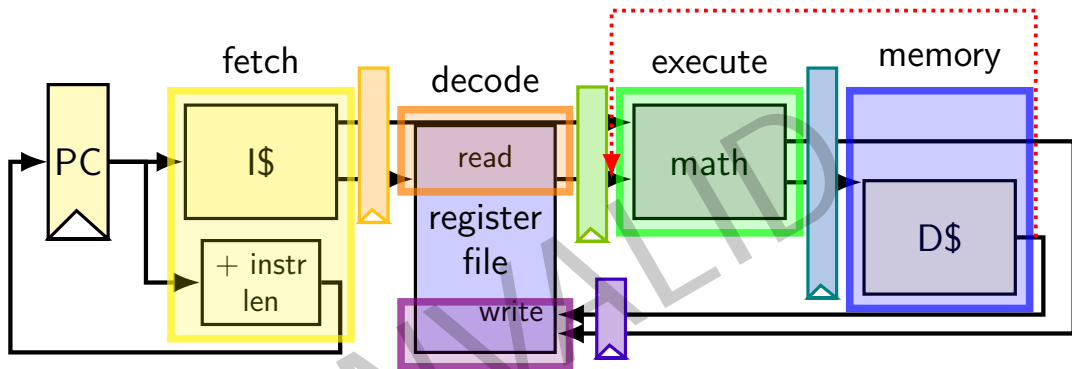
combine stalling and forwarding to resolve hazard

assumption in diagram: hazard detected in subq's decode stage
(since easier than detecting it in fetch stage)

solveable problem

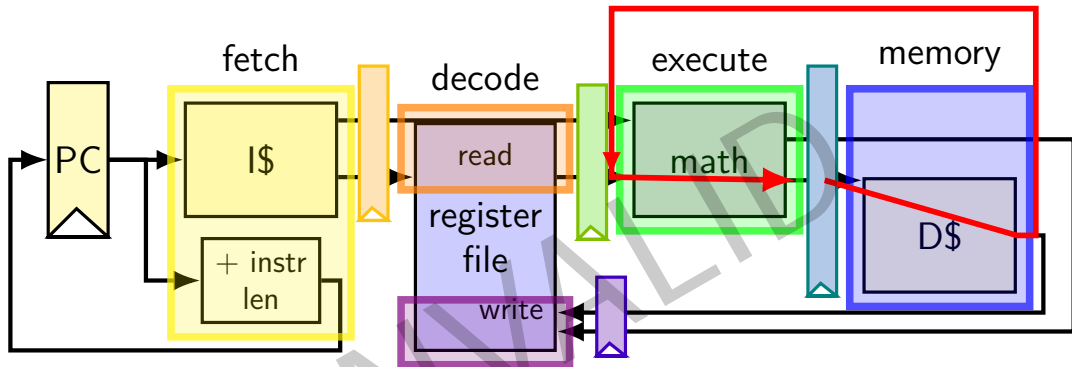
	cycle #	0	1	2	3	4	5	6	7	8
movq 0(%rax), %rbx		F	D	E	M	W				
movq %rbx, 0(%rcx)			F	D	E	M	W			

why can't we...



clock cycle needs to be long enough
to go through data cache AND
to go through math circuits!
(which we were trying to avoid by putting them in separate stages)

why can't we...



clock cycle needs to be long enough
to go through data cache AND
to go through math circuits!
(which we were trying to avoid by putting them in separate stages)

hazards versus dependencies

dependency — X needs result of instruction Y?

has potential for being messed up by pipeline
(since part of X may run before Y finishes)

hazard — will it not work in some pipeline?

before extra work is done to “resolve” hazards

multiple kinds: so far, *data hazards*, *control hazards*

ex.: dependencies and hazards (1)

addq **%rax,** **%rbx**

subq **%rax,** **%rcx**

movq **\$100,** **%rcx**

addq **%rcx,** **%r10**

addq **%rbx,** **%r10**

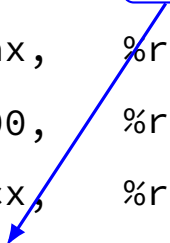
where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

ex.: dependencies and hazards (1)

addq	%rax,	%rbx
subq	%rax,	%rcx
movq	\$100,	%rcx
addq	%rcx,	%r10
addq	%rbx,	%r10



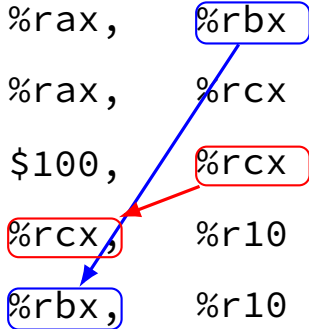
where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

ex.: dependencies and hazards (1)

addq	%rax,	%rbx
subq	%rax,	%rcx
movq	\$100,	%rcx
addq	%rcx,	%r10
addq	%rbx,	%r10



where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

ex.: dependencies and hazards (1)

addq	%rax,	%rbx
subq	%rax,	%rcx
movq	\$100,	%rcx
addq	%rcx,	%r10
addq	%rbx,	%r10

The diagram illustrates data dependencies and hazards between five instructions. A blue line connects the `%rbx` operand of the first instruction (`addq %rax, %rbx`) to the `%rbx` operand of the fifth instruction (`addq %rbx, %r10`), indicating a true data dependency. A red line connects the `%rcx` operand of the third instruction (`movq $100, %rcx`) to the `%rcx` operand of the fourth instruction (`addq %rcx, %r10`), indicating a true data dependency. A red arrow points from the `%r10` operand of the fourth instruction to the `%r10` operand of the fifth instruction, indicating a write-after-read hazard.

where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

	<i>// 4 stage</i>	<i>// 5 stage</i>
addq %rax, %r8	<i>//</i>	<i>// W</i>
subq %rax, %r9	<i>// W</i>	<i>// M</i>
xorq %rax, %r10	<i>// EM</i>	<i>// E</i>
andq %r8, %r11	<i>// D</i>	<i>// D</i>

pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

	<i>// 4 stage</i>	<i>// 5 stage</i>
<code>addq %rax, %r8</code>	<i>//</i>	<i>// W</i>
<code>subq %rax, %r9</code>	<i>// W</i>	<i>// M</i>
<code>xorq %rax, %r10</code>	<i>// EM</i>	<i>// E</i>
<code>andq %r8, %r11</code>	<i>// D</i>	<i>// D</i>

(assuming register file does not read while writing)

`addq/andq` is hazard with 5-stage pipeline

`addq/andq` is **not** a hazard with 4-stage pipeline

pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

	<i>// 4 stage</i>	<i>// 5 stage</i>
<code>addq %rax, %r8</code>	<i>//</i>	<i>// W</i>
<code>subq %rax, %r9</code>	<i>// W</i>	<i>// M</i>
<code>xorq %rax, %r10</code>	<i>// EM</i>	<i>// E</i>
<code>andq %r8, %r11</code>	<i>// D</i>	<i>// D</i>

more hazards with more pipeline stages

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

result only available near end of second execute stage

where does forwarding, stalls occur?

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8
(1) <code>addq %rcx, %r9</code>		F	D	E1	E2	M	W			
(2) <code>addq %r9, %rbx</code>										
(3) <code>addq %rax, %r9</code>										
(4) <code>movq %r9, 8(%rbx)</code>										
(5) <code>movq %rcx, %r9</code>										

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8
<code>addq %rcx, %r9</code>		F	D	E1	E2	M	W			
<code>addq %r9, %rbx</code>										
<code>addq %rax, %r9</code>										
<code>movq %r9, 8(%rbx)</code>										

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8
<code>addq %rcx, %r9</code>		F	D	E1	E2	M	W			
<code>addq %r9, %rbx</code>			F	D	E1	E2	M	W		
<code>addq %rax, %r9</code>				F	D	E1	E2	M	W	
<code>movq %r9, 8(%rbx)</code>					F	D	E1	E2	M	W

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8	
addq %rcx, %r9		F	D	E1	E2	M	W				
addq %r9, %rbx			F	D	E1	E2	M	W			
addq %r9, %rbx			F	D	D	E1	E2	M	W		
addq %rax, %r9				F	D	E1	E2	M	W		
addq %rax, %r9				F	F	D	E1	E2	M	W	
movq %r9, 8(%rbx)					F	D	E1	E2	M	W	
movq %r9, 8(%rbx)						F	D	E1	E2	M	W

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8	
addq %rcx, %r9		F	D	E1	E2	M	W				
addq %r9, %rbx			F	D	E1	E2	M	W			
addq %r9, %rbx			F	D	D	E1	E2	M	W		
addq %rax, %r9				F	D	E1	E2	M	W		
addq %rax, %r9				F	F	D	E1	E2	M	W	
movq %r9, 8(%rbx)					F	D	E1	E2	M	W	
movq %r9, 8(%rbx)						F	D	E1	E2	M	W

exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

	<i>cycle #</i>	0	1	2	3	4	5	6	7	8		
addq %rcx, %r9		F	D	E1	E2	M	W					
addq %r9, %rbx			F	D	E1	E2	M	W				
addq %r9, %rbx			F	D	D	E1	E2	M	W			
addq %rax, %r9				F	D	E1	E2	M	W			
addq %rax, %r9				F	F	D	E1	E2	M	W		
movq %r9, 8(%rbx)					F	D	E1	E2	M	W		
movq %r9, 8(%rbx)						F	D	E1	E2	M	W	
movq %rcx, %r9							F	D	E1	E2	M	W

backup slides

network security summary (1)

communicating securely with math

- secret value (shared key, public key) that attacker can't have

- symmetric: shared keys used for (de)encryption + auth/verify; fast

- asymmetric: public key used by any for encrypt + verify; slower

- asymmetric: private key used by holder for decrypt + sign; slower

protocol attacks — repurposing encrypt/signed/etc. messages

certificates — verifiable forwarded public keys

key agreement — for generated shared-secret “in public”

- publish key shares from private data

- combine private data with key share for shared secret

network security summary (2)

TLS: combine all cryptography stuff to make “secure channel”

(things we probably didn't get to:)

denial-of-service — attacker just disrupts/overloads (not subtle)

firewalls

random numbers

need a lot of keys that no one else knows

common task: choose a *random* number

question: what does *random* mean here?

cryptographically secure random numbers

security properties we might want for random numbers:

attacker cannot guess (part of) number better than chance

knowing prior 'random' numbers shouldn't help predict next 'random' numbers

compromising machine now shouldn't reveal older random numbers

exercise: how to generate?

/dev/urandom

Linux kernel random number generator

collects “entropy” from hard-to-predict events

- e.g. exact timing of I/O interrupts

- e.g. some processor's built-in random number circuit

turned into as many random bytes as you want

turning 'entropy' into random bytes

lots of ways to do this; one (rough/incomplete) idea:

internal variable *state*

to add 'entropy'

$\text{state} \leftarrow \text{SecureHash}(\text{state} + \text{entropy})$

to extract value:

$\text{random bytes} \leftarrow \text{SecureHash}(1 + \text{state})$

give bytes that can't be reversed to compute state

$\text{state} \leftarrow \text{SecureHash}(2 + \text{state})$

change state so attacker can't take us back to old state if compromised

key agreement and asym. encryption

can construct public-key encryption from key agreement

private key: generated random value Y

public key: key share generated from that Y

key agreement and asym. encryption

can construct public-key encryption from key agreement

private key: generated random value Y

public key: key share generated from that Y

$PE(\text{public key, message}) =$

- generate random value Z

- combine with public key to get shared secret

- use symmetric encryption + MAC using shared secret as keys

- output: (key share generated from Z) (sym. encrypted data) (mac tag)

key agreement and asym. encryption

can construct public-key encryption from key agreement

private key: generated random value Y

public key: key share generated from that Y

$PE(\text{public key, message}) =$
generate random value Z
combine with public key to get shared secret
use symmetric encryption + MAC using shared secret as keys
output: (key share generated from Z) (sym. encrypted data) (mac tag)

$PD(\text{private key, message}) =$
extract (key share generated from Z)
combine with private key to get shared secret, ...

exercise: forwarding paths (2)

cycle # 0 1 2 3 4 5 6 7 8

addq %r8, %r9

subq %r8, %r9

ret (goes to andq)

andq %r10, %r9

in subq, %r8 is _____ addq.

in subq, %r9 is _____ addq.

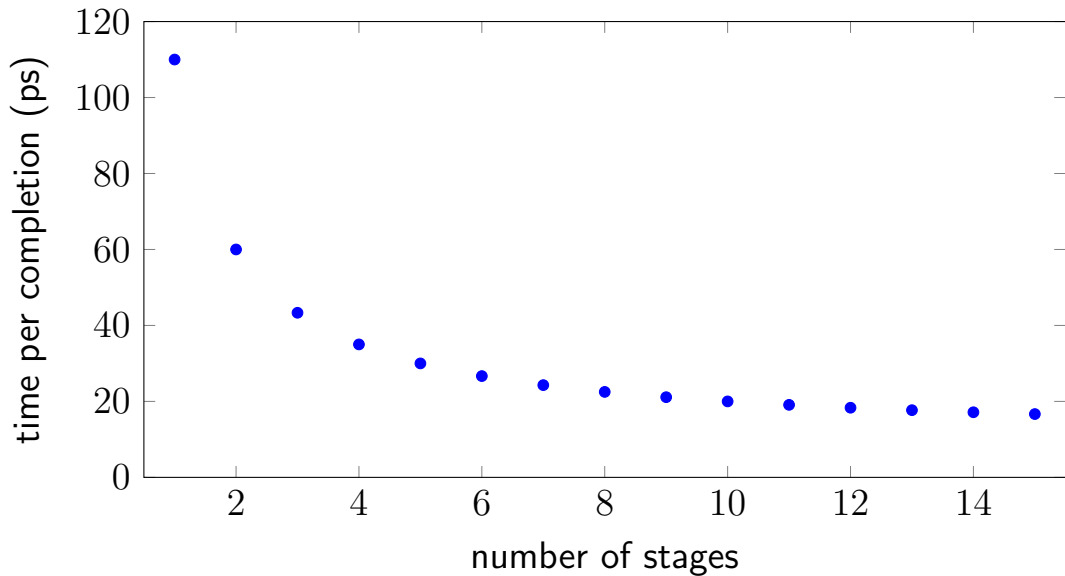
in andq, %r9 is _____ subq.

in andq, %r9 is _____ addq.

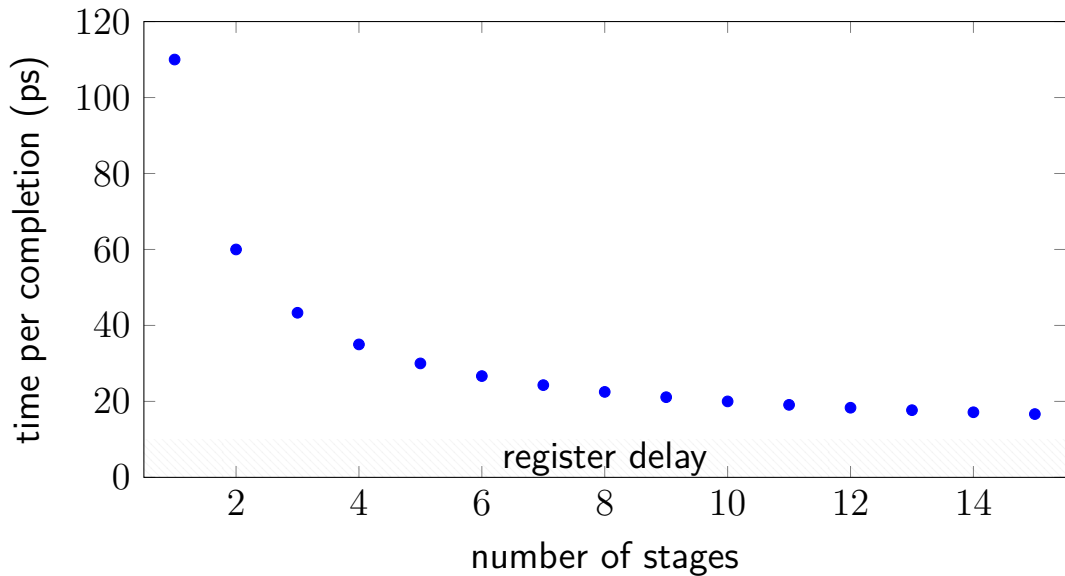
A: not forwarded from

B-D: forwarded to decode from {execute,memory,writeback} stage of

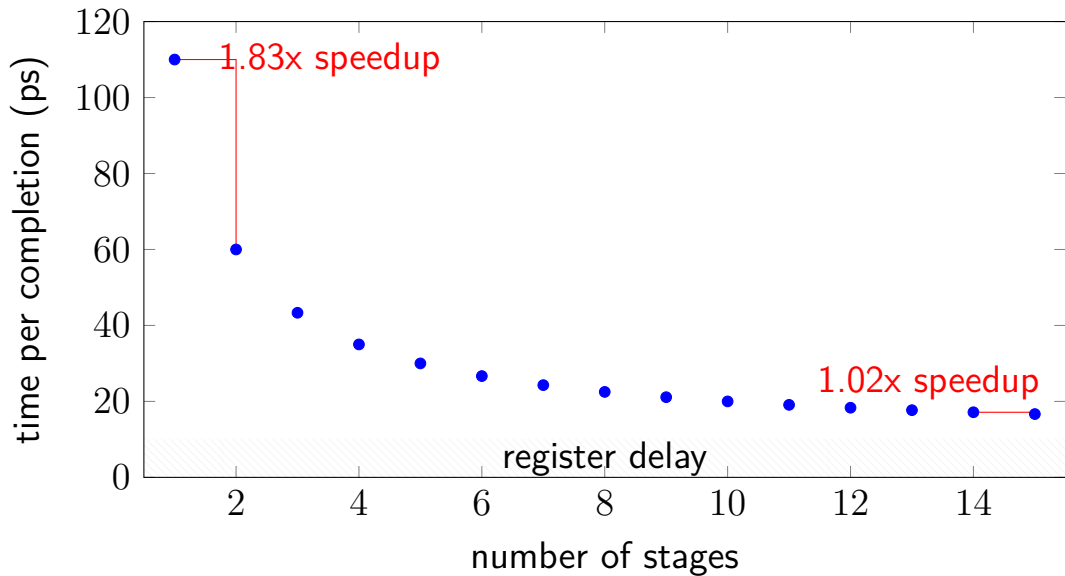
diminishing returns: register delays



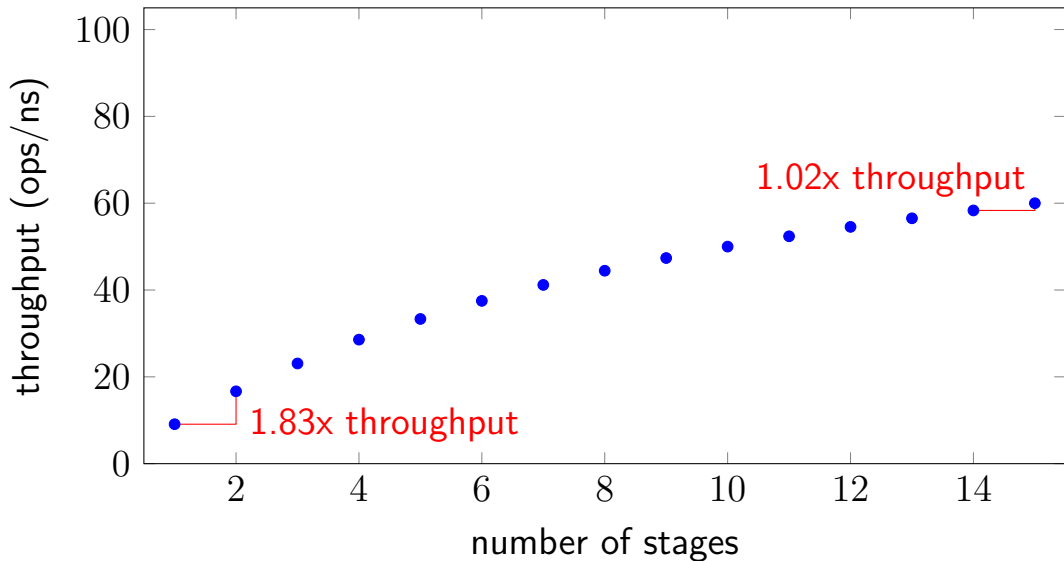
diminishing returns: register delays



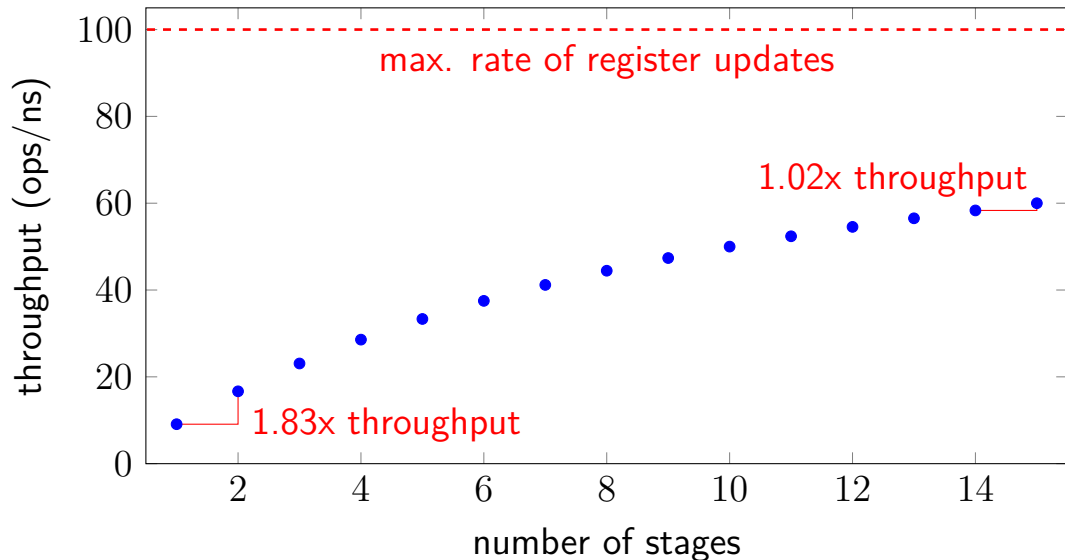
diminishing returns: register delays



diminishing returns: register delays



diminishing returns: register delays



importance of prediction

5-stage pipeline, predict not-taken versus always stall
hypothetical instruction mix

		cycles	cycles
taken jXX	3%	(predict)	(stall)
non-taken jXX	5%	not-taken)	no predict)
others	92%	1*	1*

importance of prediction

5-stage pipeline, predict not-taken versus always stall
hypothetical instruction mix

		cycles	cycles
taken jXX	3%	(predict)	(stall)
non-taken jXX	5%	not-taken)	no predict)
others	92%	1*	1*

prediction and OOO

deeper pipeline — much higher misprediction penalty